

Техническое Задание: Проектирование и Реализация Системы GraphRAG с Объяснимым Искусственным Интеллектом (XAI) на базе GNN для Здравоохранения

1. Введение и Концептуальная Основа Проекта

1.1. Контекст Проблемы и Цели

Современные системы поддержки принятия врачебных решений (CDSS) сталкиваются с критической дилеммой: необходимость использования огромных массивов неструктурированных медицинских знаний (клинические рекомендации, протоколы, истории болезней) и недопустимость стохастических ошибок, известных как "галлюцинации", присущих большим языковым моделям (LLM). Традиционный подход RAG (Retrieval-Augmented Generation), основанный исключительно на векторном поиске, демонстрирует ограничения в медицинском домене. Векторный поиск эффективно находит семантически близкие тексты, но часто игнорирует сложные, многоступенчатые причинно-следственные связи, критически важные для медицины (например, косвенное взаимодействие лекарств через метаболические пути). Данное техническое задание описывает разработку архитектуры **GraphRAG** (Graph Retrieval-Augmented Generation), которая объединяет детерминизм графов знаний (Knowledge Graphs, KG) с гибкостью генеративного ИИ. Ключевым требованием является внедрение **Объяснимого ИИ (Explainable AI, XAI)** через использование Графовых Нейронных Сетей (GNN). Система должна не только предоставлять ответ, но и визуализировать подграф "рассуждений", показывая, какие именно сущности и связи повлияли на вывод модели.

1.2. Выбор Технологического стека

Для обеспечения промышленной надежности, строгой типизации и соответствия корпоративным стандартам разработки выбран следующий стек:

- **Платформа приложений:** .NET 9/10 (C#) с использованием **Microsoft Semantic Kernel** для оркестрации агентов.
- **База данных:** PostgreSQL 18. Выбор обусловлен необходимостью конвергенции реляционных, графовых и векторных данных в едином пространстве памяти и транзакций, что устраняет проблемы синхронизации полиглотных архитектур.
- **Графовый движок:** Apache AGE (расширение для PostgreSQL) для поддержки протокола openCypher.
- **ML-инференс:** ONNX Runtime для запуска GNN моделей непосредственно внутри.NET процесса.

2. Архитектура Данных: Единая Плоскость PostgreSQL 18

2.1. Обоснование Использования PostgreSQL 18

В версии PostgreSQL 18 (релиз сентябрь 2025 года) внедрены фундаментальные изменения, критически важные для высоконагруженных систем GraphRAG. В частности, новая подсистема асинхронного ввода-вывода (Asynchronous I/O) позволяет значительно ускорить сканирование векторных индексов и графовых обходов, которые традиционно являются I/O-интенсивными операциями. Кроме того, нативная поддержка UUIDv7 обеспечивает сортируемость уникальных идентификаторов по времени, что критично для хронологического хранения медицинских событий (истории болезни, назначений). Система должна реализовывать гибридную модель хранения, где:

- 1. **Векторные данные (pgvector):** Хранят эмбединги неструктурированных текстов (выписные эпикризы, заметки врачей).
- 2. **Графовые данные (Apache AGE):** Хранят структурированные медицинские знания (онтологии, связи сущностей).
- 3. **Реляционные данные:** Хранят административную информацию (демография пациентов, биллинг).

2.2. Спецификация Расширений и Совместимости

Программист должен обеспечить корректную сборку и интеграцию расширений, учитывая новизну PostgreSQL 18.

Таблица 1. Требования к расширениям базы данных

Расширение	Версия	Назначение	Особенности реализации в PG18
Apache AGE	1.7+ (Dev)	Графовая база данных, поддержка openCypher.	Требуется сборка из исходного кода. В PG18 может потребоваться патчинг Makefile и флагов компиляции, так как официальная поддержка часто отстает от релизов ядра PG.
pgvector	0.8+	Векторный поиск (HNSW, IVFFlat).	Использование типа данных halfvec (float16) для экономии памяти при больших размерностях эмбедингов.
pg_trgm	Native	Нечеткий текстовый поиск.	Используется для Entity Linking

Расширение	Версия	Назначение	Особенности реализации в PG18
			(привязки сущностей из текста запроса к узлам графа).
uuid-ossf	Native	Генерация UUID.	Заменяется на нативную функцию uuidv7() в PG18 для генерации ключей, сортируемых по времени.

2.3. Схема Гибридного Поиска

Программист должен реализовать хранимые процедуры на PL/pgSQL, которые инкапсулируют логику гибридного поиска, минимизируя round-trip задержки между приложением и БД.

Пример логики запроса hybrid_search_context:

1. **Вход:** Вектор запроса ($V_{\{query\}}$), список извлеченных сущностей ($E_{\{list\}}$).
2. **Шаг 1 (Векторный):** Выполнить поиск KNN по таблице embeddings с использованием оператора $\leq$ (косинусное расстояние).
3. **Шаг 2 (Графовый):** Использовать сущности $E_{\{list\}}$ как стартовые узлы (Entry Points) в графе Apache AGE. Выполнить обход на 2-3 хопа (hops) для поиска связанных концептов (например, "Лекарство" -> "Побочный эффект" -> "Симптом").
4. **Слияние (Fusion):** Объединить результаты. Если документ, найденный векторным поиском, имеет ссылку на узел графа, повысить релевантность (Reranking).

Техническое требование: Реализовать функцию с использованием CTE (Common Table Expressions) для параллельного выполнения графовой и векторной части плана запроса, пользуясь улучшениями планировщика PG18.

3. Спецификация Графовой Модели и Интеграция FHIR

3.1. Онтологическая Модель

Основой графа знаний должен стать стандарт HL7 FHIR (Fast Healthcare Interoperability Resources). Прямая трансляция JSON-структур FHIR в графовые узлы требует нормализации.

Правила маппинга (Mapping Rules):

1. **Ресурсы как Узлы:** Каждый ресурс FHIR (Patient, Observation, MedicationRequest) преобразуется в узел с меткой, соответствующей типу ресурса.
2. **Ссылки как Ребра:** Поля типа Reference (например, subject в Condition) преобразуются в направленные ребра.
 - Condition ----> Patient
 - MedicationRequest ----> CarePlan
3. **Терминология:** Коды SNOMED CT, LOINC и RxNorm должны быть выделены в отдельные узлы Concept, чтобы обеспечить связность графа между разными

пациентами.

- *Неправильно:* Хранить код "J45.9" как строковое свойство узла Condition пациента А и пациента Б.
- *Правильно:* Пациент А ----> Узел(Condition) ----> Узел(Concept {code: "J45.9"})
<---- Узел(Condition) <---- Пациент Б. Это позволяет GNN обучаться на паттернах заболеваний.

3.2. Стратегия Ingestion (ETL)

Разработать ETL-конвейер на C#, который:

1. Парсит входящие FHIR Bundle (JSON).
2. Использует библиотеку Hl7.Fhir.R4.
3. Генерирует команды Cypher CREATE или MERGE.

Критическое требование: Использовать MERGE для узлов онтологии (лекарства, диагнозы), чтобы избежать дублирования, и CREATE для событийных данных (конкретное измерение температуры), используя UUIDv7 в качестве первичных ключей.

4. Программная Реализация Backend на.NET и Semantic Kernel

4.1. Архитектура Агента

Использовать **Microsoft Semantic Kernel (SK)** для создания агента, способного планировать выполнение запроса. Агент не должен быть жестко запрограммированным алгоритмом, а должен использовать "Планировщик" (Planner) для выбора инструментов.

Необходимые Плагины (Kernel Plugins):

1. **GraphQueryPlugin:**
 - *Функция:* ExecuteCypher(string query)
 - *Назначение:* Выполнение прямых запросов к Apache AGE.
 - *Валидация:* Встроенная проверка на SQL-инъекции и ограничение глубины обхода (максимум 3 хопа) для предотвращения перегрузки БД.
2. **VectorMemoryPlugin:**
 - *Функция:* SearchNotes(string queryEmbedding)
 - *Назначение:* Поиск по pgvector. Необходимо реализовать интерфейс IMemoryStore Semantic Kernel поверх Postgres.
3. **TerminologyPlugin:**
 - *Функция:* NormalizeEntity(string rawText)
 - *Назначение:* Преобразование пользовательского ввода (например, "Advil") в нормализованный код (RxNorm code for Ibuprofen), используя внешние API или локальный справочник.

4.2. Реализация GraphRAG Потoka (Workflow)

Программист должен реализовать следующий алгоритм обработки запроса в классе GraphRagService.cs:

1. **Извлечение сущностей (NER):** Использовать небольшую локальную модель

(например, через ML.NET или ONNX BERT) для выделения медицинских сущностей из промпта пользователя.

2. **Генерация подграфа:**

- Сформировать Cypher-запрос, который находит узлы, соответствующие извлеченным сущностям.
- Расширить подграф (Graph Expansion), включив соседей.

3. **Векторное обогащение:** Параллельно выполнить поиск по векторной базе для нахождения контекстуальных заметок.

4. **Инференс GNN:** Передать структуру подграфа в модуль GNN (см. Раздел 5) для оценки важности узлов.

5. **Формирование Промпта:** Отфильтровать узлы с низким весом GNN.

Сериализовать оставшийся подграф (в формате JSON или тройках RDF) и добавить его в системный промпт LLM.

- *Шаблон:* "Используй следующие факты из графа знаний для ответа: {GraphContext}. Используй следующие заметки для контекста: {VectorContext}".

5. Подсистема Машинного Обучения: GNN и ONNX Runtime

5.1. Роль GNN в RAG

В данной архитектуре Графовая Нейронная Сеть (GNN) выполняет роль **Ранжировщика (Scorer/Reranker)**. Она оценивает вероятность того, что связь между двумя узлами в извлеченном подграфе является релевантной для текущего клинического контекста. Это позволяет отсеять "шум" (случайные связи) и сосредоточиться на значимых путях.

5.2. Модель и Обучение

- **Архитектура:** Использовать **GraphSAGE** или **GAT (Graph Attention Network)**. Эти архитектуры поддерживают индуктивное обучение (работают с новыми узлами/пациентами, не виденными при обучении).
- **Экспорт:** Модель обучается на Python (PyTorch Geometric), но должна быть экспортирована в формат **ONNX** для исполнения в .NET среде.
 - *Требование к экспорту:* Входы модели должны быть динамическими (dynamic axes), так как размер подграфа (количество узлов и ребер) варьируется от запроса к запросу.

5.3. Инференс в C# (Microsoft.ML.OnnxRuntime)

Программист должен реализовать преобразование данных из Apache AGE (JSON/Agtype) в тензоры ONNX (DenseTensor).

Спецификация Тензоров:

1. **node_features (Input):** Тензор размерности $N \times D$, где N — количество узлов в подграфе, D — размерность вектора признаков узла (эмбединг названия + тип).
2. **edge_index (Input):** Тензор размерности $[2, E]$, где E — количество ребер. Содержит пары индексов (source, target) в формате Int64 (long).

- **Важно:** Apache AGE возвращает глобальные ID графа. Для GNN их необходимо переиндексировать в локальный диапазон [0, N-1] для текущего тензора.

Пример кода подготовки данных (C#):

```
// Использование Span<T> для высокой производительности и минимизации
// аллокаций
var nodeFeatures = new DenseTensor<float>(new { numNodes, featureDim
});
var edgeIndices = new DenseTensor<long>(new { 2, numEdges });

// Заполнение тензоров данными из AGE...
// Вызов сессии ONNX
using var results = session.Run(new List<NamedOnnxValue>
{
    NamedOnnxValue.CreateFromTensor("x", nodeFeatures),
    NamedOnnxValue.CreateFromTensor("edge_index", edgeIndices)
});
```

6. Механизмы Объяснимости (Explainability/XAI)

6.1. Методология GNNExplainer

Ключевое требование — предоставить врачу объяснение. Для этого используется адаптация подхода **GNNExplainer** или **GNN-XAR**. Система должна выделить подграф, который максимизировал уверенность модели в предсказании.

6.2. Техническая Реализация

Поскольку полный итеративный запуск GNNExplainer для каждого запроса слишком медленный (секунды/минуты), необходимо использовать **метод внимания (Attention Weights)** из модели GAT (Graph Attention Network).

1. В процессе инференса извлечь веса внимания (attention coefficients) из последнего слоя GNN.
2. Нормировать веса. Ребра с наибольшим весом считаются "объяснением".
3. **Формат вывода:** API должно возвращать JSON-объект, содержащий не только текст ответа, но и список ID ребер с их весами важности.

Пример JSON-ответа для UI:

```
{
  "response": "Назначение препарата X противопоказано...",
  "explanation": {
    "graph_visualization": {
      "nodes": [...],
      "edges":
    }
  }
}
```

Фронтенд должен отрисовывать эти ребра жирными красными линиями, визуально демонстрируя путь рассуждения.

7. Безопасность, RLS и Соответствие HIPAA

7.1. Row Level Security (RLS) в Графах

Медицинские данные требуют строгой изоляции (Multi-tenancy). В PostgreSQL RLS работает на уровне таблиц. Однако Apache AGE хранит данные графа в специфических таблицах схемы ag_catalog.

- **Проблема:** Прямое применение RLS к таблицам ag_catalog может нарушить внутреннюю логику графа.
- **Решение:** Реализовать RLS на уровне **функций доступа**. Все запросы к графу должны обязательно включать предикат принадлежности пациента к текущему арендатору (клинике) или врачу.
 - *Паттерн запроса:* MATCH (p:Patient {tenant_id: \$tid})-[*]->(target)...
 - Программист должен создать middleware в .NET, которое автоматически injectрует tenant_id во все Cypher-запросы перед их выполнением.

7.2. Аудит и Шифрование

1. **Шифрование:** Включить шифрование диска (TDE) и принудительное использование SSL/TLS 1.3 для соединений Npgsql.
2. **Аудит:** Использовать расширение pgaudit для логирования доступа к таблицам с персональными данными (PII). Поскольку запросы GraphRAG могут быть сложными, необходимо логировать текст Cypher-запроса для последующего анализа безопасности.

8. План Разработки и Развертывания (Roadmap)

Разработка разбивается на фазы для минимизации рисков интеграции новых технологий (PG18).

Таблица 2. Фазы реализации проекта

Фаза	Задача	Ключевые Результаты	Технические Риски
I	Инфраструктура	Сборка Docker-образа PG18 + AGE + pgvector. Развертывание базовой схемы БД.	Несовместимость AGE с PG18 API (требуется С-разработки/патчинг а).
II	Backend Core	Реализация .NET сервиса, Semantic Kernel плагинов, маппинга FHIR->Graph.	Сложность маппинга глубоко вложенных структур FHIR в плоский граф.
III	ML & GNN	Обучение GNN модели, экспорт в	Проблемы с размерностями

Фаза	Задача	Ключевые Результаты	Технические Риски
		ONNX, интеграция пайплайна инференса.	тензоров и поддержкой операторов в ONNX Runtime.
IV	GraphRAG & XAI	Реализация гибридного поиска, логики "объяснения" (Attention extraction), интеграция с LLM.	Галлюцинации LLM даже при наличии контекста (требует fine-tuning промптов).
V	Оптимизация	Настройка индексов (HNSW, GIN), кэширование подграфов, нагрузочное тестирование.	Производительность обхода графа на больших данных (>10 млн узлов).

8.1. Требования к Тестированию

- **Unit Tests:** Покрытие логики маппинга FHIR и тензорных преобразований.
- **Integration Tests:** Тесты на реальной БД (Testcontainers для PostgreSQL) для проверки корректности Cypher-запросов и RLS.
- **Factuality Tests:** Использовать отдельную LLM ("LLM-судья") для проверки того, что сгенерированный ответ не противоречит извлеченному подграфу фактов.

Заключение для Исполнителя: Данное ТЗ требует высокой квалификации в системном программировании (работа с драйверами БД, памятью.NET), понимании ML-пайплайнов (ONNX, тензоры) и специфики медицинских данных. Успешная реализация проекта создаст прецедент использования полностью прозрачного (Explainable) ИИ в критически важной сфере здравоохранения, опираясь на новейшие возможности экосистемы PostgreSQL 18.

Источники

1. Explainable Knowledge Graph Retrieval-Augmented Generation (KG-RAG) with KG-SMILE - arXiv, <https://arxiv.org/html/2509.03626v1> 2. A Survey of Graph Retrieval-Augmented Generation for Customized Large Language Models - OpenReview, <https://openreview.net/pdf?id=9FJiOMuZkr> 3. Hybrid Multi-Agent GraphRAG for E-Government: Towards a Trustworthy AI Assistant - MDPI, <https://www.mdpi.com/2076-3417/15/11/6315> 4. What is GraphRAG? - IBM, <https://www.ibm.com/think/topics/graphrag> 5. An Ontology-based-RAG System using Graph for accurate analytics | by Max.AI Platform, <https://medium.com/max-ai-tech-blog/building-a-graph-based-rag-system-using-apache-age-98e0ac7f9589> 6. PostgreSQL 18 Release Notes, <https://www.postgresql.org/docs/release/18.0/> 7. PostgreSQL 18 Released!, <https://www.postgresql.org/about/news/postgresql-18-released-3142/> 8. Going down the rabbit hole of Postgres 18 features - Xata, <https://xata.io/blog/going-down-the-rabbit-hole-of-postgres-18-features> 9. 2.4.1. Database Technology Assessment - Documentation for World Historical Gazetteer, <https://docs.whgazetteer.org/content/v4/architecture/database.html> 10. UBS unlocks advanced

AI techniques with PostgreSQL on Azure | Microsoft Community Hub, <https://techcommunity.microsoft.com/blog/adforpostgresql/ubs-unlocks-advanced-ai-techniques-with-postgresql-on-azure/4403561> 11. Hybrid search with Postgres Native BM25 and VectorChord, <https://docs.vectorchord.ai/vectorchord/use-case/hybrid-search.html> 12. How to Implement Graph RAG Using Knowledge Graphs and Vector Databases - Medium, <https://medium.com/data-science/how-to-implement-graph-rag-using-knowledge-graphs-and-vector-databases-60bb69a22759> 13. Building a job search engine with PostgreSQL's advanced search features - AWS, <https://aws.amazon.com/blogs/database/building-a-job-search-engine-with-postgresqs-advanced-search-features/> 14. Data mapping to FHIR: What you need to know before starting - Firely, <https://fire.ly/blog/data-mapping-to-fhir-what-you-need-to-know-before-starting/> 15. Standardizing Internal Data Models on FHIR - Progress Software, <https://www.progress.com/blogs/standardizing-internal-data-models-on-fhir> 16. How to create a GenAI agent using Semantic Kernel | Nearform, <https://nearform.com/digital-community/how-to-create-a-genai-agent-using-semantic-kernel/> 17. Building Smarter RAG Systems with Semantic Kernel and SQL Memory | by Denis Pashkov, <https://medium.com/@pashkovdenis/building-smarter-rag-systems-with-semantic-kernel-and-sql-memory-4713f571f6da> 18. Building LLM Applications with Kernel Memory and Redis, <https://redis.io/blog/building-llm-applications-with-kernel-memory-and-redis/> 19. Welcome - GraphRAG, <https://microsoft.github.io/graphrag/> 20. Self-Explainable Graph Neural Network for Alzheimer Disease and Related Dementias Risk Prediction: Algorithm Development and Validation Study - PubMed Central, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11263893/> 21. Graph — ONNX GraphSurgeon 0.3.11 documentation, <https://docs.nvidia.com/deeplearning/tensorrt/archives/tensorrt-803/onnx-graphsurgeon/docs/ir/graph.html> 22. ONNX Shape Inference - ONNX 1.21.0 documentation, <https://onnx.ai/onnx/repo-docs/ShapeInference.html> 23. Shapes of intermediate tensors · Issue #4580 · onnx/onnx - GitHub, <https://github.com/onnx/onnx/issues/4580> 24. Basic C# Tutorial | onnxruntime, https://onnxruntime.ai/docs/tutorials/csharp/basic_csharp.html 25. Creating a onnx model in python and running inference in C# | by rohin ramesh - Medium, <https://medium.com/@rohinfablabz/creating-a-onnx-model-in-python-and-running-inference-in-c-98be02d2518a> 26. PyTorch Cookbook (2023).pdf - NBKR Institute Of Science & Technology, [http://103.203.175.90:81/fdScript/RootOfEBooks/E%20Book%20collection%20-%202023%20-%20G/CSE%20%20IT%20AIDS%20ML/PyTorch%20Cookbook%20\(2023\).pdf](http://103.203.175.90:81/fdScript/RootOfEBooks/E%20Book%20collection%20-%202023%20-%20G/CSE%20%20IT%20AIDS%20ML/PyTorch%20Cookbook%20(2023).pdf) 27. Reliable and Faithful Generative Explainers for Graph Neural Networks - MDPI, <https://www.mdpi.com/2504-4990/6/4/139> 28. GNN-XAR: A Graph Neural Network for Explainable Activity Recognition in Smart Homes, <https://arxiv.org/html/2502.17999v1> 29. PROVEX: Enhancing SOC Analyst Trust with Explainable Provenance-Based IDS - arXiv, <https://arxiv.org/html/2512.18199v1> 30. CLARUS: An Interactive Explainable AI Platform for Manual Counterfactuals in Graph Neural Networks - bioRxiv, <https://www.biorxiv.org/content/10.1101/2022.11.21.517358v3.full.pdf> 31. Implement row-level security in Amazon Aurora MySQL and Amazon RDS for MySQL - AWS, <https://aws.amazon.com/blogs/database/implement-row-level-security-in-amazon-aurora-mysql-and-amazon-rds-for-mysql/> 32. Multi-tenant data isolation with PostgreSQL Row Level Security | AWS Database Blog, <https://aws.amazon.com/blogs/database/multi-tenant-data-isolation-with-postgresql-row-level-security/> 33. Postgres constraint for age - Stack Overflow, <https://stackoverflow.com/questions/79093108/postgres-constraint-for-age> 34. PostgreSQL 18 Security Enhancements: OAuth2, Checksums & Enterprise Compliance with DMAP AI

Automation - Newt Global Consulting LLC,
<https://newtglobal.com/blogs/postgresql-18-security-compliance-dmap-ai/> 35. Enforce row-level security with the RDS Data API | AWS Database Blog,
<https://aws.amazon.com/blogs/database/enforce-row-level-security-with-the-rds-data-api/>